COMPTE RENDU INFO4B

Indexeur de fichiers

Mehdi BENSELAMA & Yacine BEKHTAOUI

Année universitaire 2025/2026

1. Introduction 3
2. Analyse fonctionnelle 3
2.1 Description générale 3
2.2 Règles de fonctionnement 3
2.2.1 Partie Indexeur 3
2.2.2 Partie Utilisateur 4
3. Structures de données 4
3.1 Structures envisagées 4
3.2 Structures retenues 5
4. Architecture logicielle en couches 6
4.1 Couche 1 : Données (indexeur.data) 6
4.2 Couche 2 : Moteur (indexeur.moteur) 6
4.3 Couche 3 : Réseau (indexeur.reseau) 7
4.4 Couche 4 : Interface (indexeur.iu) 7
5. Description des algorithmes principaux 8
5.1 Tokenisation (extraction des mots) 8
5.2 TF-IDF (scoring de recherche) 8
5.3 Détection de doublons 9
5.4 Exploration récursive 9
5.5 Communication par pipe 10
6. Threads et synchronisation 11
6.1 Liste des threads 11
6.2 Mécanismes de synchronisation 11
6.3 Modèle producteur/consommateur 11
7. Jeux de tests 12
7.1 Test 1 : Indexation d'un répertoire 12
7.2 Test 2 : Recherche par mots-clés 12
7.3 Test 3 : Détection de doublons 12
7.4 Test 4 : Lister les documents indexés 13
7.5 Test 5 : Annotation et recherche par métadonnées 13
7.6 Test 6 : Status 13
7.7 Test 7 : Ajout de mots personnalisés 13
7.8 Test 8 : Ajout de stop-words 14
7.9 Test 9 : Connexion réseau 14
8. Répartition des tâches 16
9. Conclusion 16
10. Annexe : Utilisation d'un LLM
10.1 Code généré par le LLM 17
10.2 Texte du rapport généré par le LLM 17
10.3 Prompts utilisés 17
10.4 Ce que nous avons appris du LLM 17

1. Introduction

Ce rapport présente la réalisation du projet de l'UE Principes des Systèmes d'Exploitation Info4B.
Le sujet choisi est le **Sujet 1 :** Indexation de fichiers et recherches. L'objectif est de construire une application permettant d'indexer le contenu de documents, d'effectuer des recherches par mots-clés avec un scoring TF-IDF, de détecter les doublons et d'ajouter des métadonnées sur les fichiers.

Ce projet nous a permis de faire de la communication inter-processus notamment grâce au principe de pipe avec le programme *pdftotext* de Linux, de faire du multithreading pour l'exploration de l'arborescence en respectant les principes de concurrence (synchronized, wait) ainsi qu'une communication réseau par sockets pour l'interaction à distance.

Ce document débute par une analyse fonctionnelle du sujet, suivie de la description des structures de données utilisées. Nous détaillerons ensuite l'architecture logicielle couche par couche puis les algorithmes principaux sont expliqués.
Le rapport se termine par des jeux de tests réalisés et la répartition des tâches entre les deux membres du binôme pour finir avec une conclusion et l'utilisation du LLM.

2. Analyse fonctionnelle

2.1 Description générale

Notre programme est un indexeur de fichiers multithreading. Il fonctionne avec une architecture en 4 couches. L'indexeur parcourt récursivement un répertoire, extrait le contenu des fichiers texte, PDF et images, et l'index. L'indexeur de fichiers propose un moteur de recherche par mots clés qui utilise le scoring TF-IDF.

Le programme utilise une architecture Client-Serveur. Le serveur gère l'indexation et permet aux clients de se connecter via un socket TCP. Les clients se connectent pour effectuer des recherches. En local, le menu console permet une interaction directe avec le programme.

L'indexation se fait en parallèle grâce au modèle producteur/consommateur. Un thread explorateur parcourt les fichiers, les mets dans une file d'attente, et quatre threads indexeurs les traitent simultanément. L'index est enregistré régulièrement sur le disque par sérialisation Java pour conserver les données.

2.2 Règles de fonctionnement

2.2.1 Partie Indexeur

- **Exploration multithreading** : un thread explorateur parcourt récursivement l'arborescence des fichiers à partir d'un répertoire de base. Puis 4 threads indexeurs traitent les fichiers trouvés.
- **Indexation des mots** : tous les mots dans un document sont extraits et enregistrés puis comptabilisés dans la fiche du document correspondant.
- **Conversion des PDF** : les PDF sont convertis en texte via le *pdftotext* en utilisant ProcessBuilder.
- **Extraction donnée EXIF** : pour les images, les métadonnées de base (nom, type) sont récupérées et stockées dans la fiche du document.

- **Sauvegarde** : l'index est sauvegardé automatiquement toutes les 30 secondes sur disque par sérialisation Java.
- **Détection des doublons** : un checksum est calculé pour chaque fichier, permettant de détecter les fichiers identiques.
- **Scoring TF-IDF** : les résultats de recherche sont classés par pertinence par score TF-IDF.

2.2.2 Partie Utilisateur

- **Recherche par contenu :** L'utilisateur peut rechercher des fichiers en saisissant un ou plusieurs mots-clés. Le programme retourne une liste de fichiers pertinents triés par score TF-IDF .
- **Recherche par métadonnées :** L'utilisateur peut rechercher des fichiers selon leurs métadonnées en donnant une clé et une valeur. Par exemple, il peut chercher tous les fichiers dont l'auteur est "TOTO" en saisissant la clé "auteur" et la valeur "TOTO". Le programme retourne donc tous les fichiers dont la métadonnée correspond exactement à la recherche.
- **Annotation de fichiers :** L'utilisateur peut enrichir un fichier en lui ajoutant des métadonnées personnalisées
- **Ajout de stop-words et termes personnalisés:** L'utilisateur peut ajouter des mots à ignorer lors de l'indexation, ainsi que ajouter des mots à forcer dans l'indexation
- **Connexion distante via socket TCP:** Un client peut se connecter à l'indexeur depuis une autre machine et effectuer des recherches à distance.
- **Récupération de fichiers à distance:**Le client connecté à distance peut récupérer une copie d'un fichier sur sa machine.

3. Structures de données

3.1 Structures envisagées

Contrairement aux Hashtable, la Hashmap est très rapide mais n'est pas protégée contre les accès en simultanés.

Donc pour ce projet qui utilise un principe de multithreading il a été préférable d'utiliser les Hashtable plutôt car elles sont synchronisées par défaut et assurent la cohérence des données en empêchant les conflits d'accès simultanés.

Concernant la file d'attente des fichiers à explorer, nous avons opté pour l'utilisation de la LinkedList plutôt que l'ArrayList car elle permet la gestion d'une file FIFO puisque l'ajout à la fin et le retrait au début se font en temps constant; contrairement à l'ArrayList qui doit être réorganisé à chaque modification en début de chaîne, ce qui ralentit naturellement le programme lorsqu'il y a beaucoup de fichiers.

De plus, l'utilisation de HashSet est idéale pour les ensembles stop-words et la détection doublons car elle permet de stocker des éléments uniques sans ordre particulier ce qui offre une recherche quasi instantanée en complexité O(1), ce qui est crucial dans un jeu de données conséquent contenant plusieurs milliers de mots.

3.2 Structures retenues

Structure	Type Java	Rôle	gestion de la concurrence
FileQueue	LinkedList<File>	File producteur/consommateur	synchronized + wait/notify
index	Hashtable<String, HashSet<String>>	Index inversé mot → fichiers	Hashtable (synchronized par défaut)
catalogue	Hashtable<String, FicheDocument>	Chemin → fiche document	Hashtable
doublons	Hashtable<String, HashSet<String>>	Checksum → fichiers identiques	Hashtable
frequenceTermes	Hashtable<String, Integer>	Mot → nb occurrences par doc	Accès unique par ThreadIndexeur via FicheDocument
stopWords	HashSet<String>	Mots à ne pas indexer	Méthodes synchronized dans IndexInverse

- **La FileQueue**
  Nous avons implémenté la FileQueue selon le modèle «Producteur-Consommateur» qui permet de stocker les fichiers trouvés par le thread exploration, en attendant bien sûr qu'un thread de traitement soit libre pour les indexer.

- **L'index**
  L'index inversé associe chaque mot-clé à un ensemble (HashSet) de chemins de fichiers, ce qui évite donc de reparcourir tout le disque inutilement lors d'une recherche utilsateur car le système pointe directement vers les documents concernés.

- **Le Catalogue**
  Le Catalogue associe le chemin unique d'un fichier *(exemple: C:\Users\Admin\Desktop\Info4B\fi\src\fileindexer\data)* à sa FicheDocument, permettant de regrouper ses métadonnées et ses statistiques.

- **Les Doublons**
  Pour détecter les doublons, nous utilisons une table qui associe une empreinte digitale à une liste de chemins de fichiers. L'empreinte est calculée en additionnant les valeurs de tous les octets du fichier, puis en convertissant cette somme en hexadécimal. Par exemple, si deux fichiers ont une somme identique (comme `2a3b4c5d`), ils sont considérés comme identiques (doublons). Si les sommes diffèrent alors les fichiers sont très probablement différents.

- **Les FréquencesTermes**
  La structure FrequenceTermes compte combien de fois chaque mot apparaît dans un document pour donner un score de recherche grâce via la mesure TF-IDF où elle stocke l'occurrence de chaque mot et en détermine la pertinence

- **StopWords**
  L'ensemble stopWords permet de filtrer les termes "négligeables" (articles, conjonctions de coordinations) durant l'indexation pour permettre d'alléger la base de données et donc d'améliorer la pertinence des recherches.

4. Architecture logicielle en couches

4.1 Couche 1 : Données (indexeur.data)

Cette couche regroupe les structures de données nécessaires au stockage et à la manipulation des informations indexées.

Classe	Méthodes essentielles	Rôle
FileQueue	ajouter(), retirer(), fermer()	Gère la file d'attente des fichiers à traiter via un mécanisme de synchronisation (wait/notify)
FicheDocument	ajouterMot(), getNombreTotal()	Stocke les métadonnées d'un fichier et les statistiques de fréquences des mots
IndexInverse	ajouterDocument(), rechercher(), sauvegarder(), charger()	Relie les mots aux fichiers et gère la sauvegarde de l'index sur le disque (sérialisation)
ResultatRecherche	compareTo(), getScore()	Stocke le score TF-IDF d'un fichier trouvé pour permettre un tri par pertinence

Celle-ci est la base de l'application. Elle contient les structures telles que Hashtable et HashSet qui, comme nous l'avons vu précédemment, garantissent l'intégrité des données face aux accès concurrents des threads.

4.2 Couche 2 : Moteur (indexeur.moteur)

Le moteur quant à lui gère la logique métier et gère le multithreading pour l'exploration et l'indexation.

Classe	Méthodes essentielles	Rôle
Explorateur	run(), explorer()	Parcourt l'arborescence pour trouver les fichiers et remplir la file d'attente
ThreadIndexeur	run(), indexerTexte(), indexerPdf()	Extrait les mots des fichiers (via pipes pour PDF) et les ajoute à l'index)
MoteurIndexation	lancer(), attendre()	Ordonnance et gère le cycle de vie des threads d'exploration et d'indexation

Ici, l'Explorateur agit comme producteur, tandis que les ThreadIndexeur agissent comme consommateurs. La communication avec les outils externes comme *pdftotext* se fait ici via des pipes.

4.3 Couche 3 : Réseau (indexeur.reseau)

Cette couche gère la communication distante entre l'utilisateur et l'indexeur via des sockets.

Classe	Méthodes essentielles	Rôle
Serveur	run(), arreter()	Attend les connexions sur un port et crée un thread pour chaque nouveau client
Client	run(), traiterCommande()	Thread dédié à un client pour recevoir ses ordres et lui renvoyer les résultats

Le ServeurIndex attend que quelqu'un se connecte, puis dès qu'un Client arrive, il créer une ConnexionClient. L'avantage d'utiliser des threads ici est que le serveur peut répondre à plusieurs personnes en même temps sans être bloqué par une recherche un peu longue.

4.4 Couche 4 : Interface (indexeur.iu)

C'est la partie visible du programme, celle où l'utilisateur tape ses commandes.

Classe	Méthodes essentielles	Rôle
ProgrammePrincipal	demarrer(), boucleCommandes(), main()	Démarre l'application et affiche le menu pour que l'utilisateur puisse agir

Cette couche sert de lien entre l'humain et la machine. Elle reçoit les commandes du clavier (comme search ou status) et appelle les bonnes fonctions dans les couches du dessous pour obtenir le résultat.

5. Description des algorithmes principaux

5.1 Tokenisation (extraction des mots)

Cet algorithme découpe le texte pour isoler les mots utiles :
Lecture et Découpage

On utilise un *BufferedReader* pour lire le fichier ligne par ligne puis grâce à un StringTokenizer on sépare les mots en fonction des espaces et de la ponctuation.
Pour chaque mot extrait, s'il est trop court (de longueurs inférieure ou égale à 2), on vérifie s'il appartient au *HashSet* des stopwords puis sera ignoré si c'est le cas.
Si le mot est conservé, on met à jour sa fréquence dans la *Hashtable* du document.

```
private void indexerTexte(File f, FicheDocument fiche) throws IOException {
    BufferedReader reader = new BufferedReader(new FileReader(f));
    String ligne = reader.readLine();
    while(ligne!=null)
    {
        ligne = ligne.toLowerCase();
        //découpage de la ligne en mots à l'aide de délimiteurs
        StringTokenizer st = new StringTokenizer(ligne, delim: " ,;.:+-=* /{}?!()[]\\t\\n\\r");
        //parcours tous les mots de la ligne
        while(st.hasMoreTokens())
        {
            String mot = st.nextToken();
            //on ignore les mots trop courts
            if(mot.length()<2) continue;
            if(index.estStopWord(mot)) continue;
            //ajout du mot filtré dans la fiche du document pour l'indexation
            fiche.ajouterMot(mot);
        }
        ligne = reader.readLine();
    }
    reader.close();
}
```

5.2 TF-IDF (scoring de recherche)

Cet algorithme permet de trier les documents par pertinence. Pour évaluer la pertinence d'un document par rapport à une recherche, nous utilisons la mesure standard TF-IDF comme suggérée dans l'énoncé du projet.

- **TF (Term Frequency) :** Calcule si le mot est très présent dans ce document précis
- **IDF :** Calcule si le mot est rare par rapport à tous les documents du disque
- **Score final :** On multiplie les deux. Si un mot est très présent dans un seul document alors cela donnera un score élevé à ce document.

5.3 Détection de doublons

Afin d'éviter de comparer les fichiers octet par octet, nous avons choisi d'utiliser une empreinte numérique **(checksum)** pour chaque fichier.

- L'algorithme lit le contenu du fichier et calcule une somme des octets, convertie ensuite en une chaîne de caractères hexadécimaux.
- Si deux fichiers possèdent la même empreinte, ils sont considérés comme des doublons.
- De plus, ces empreintes sont stockées dans une Hashtable afin de permettre une comparaison rapide.

```java
private String calculerChecksum(File f) {
    try {
        //lecture du contenu du fichier sous forme de tableau de bytes
        byte[] data = Files.readAllBytes(f.toPath());
        //on initialise la somme qui servira à crer une empreinte du fichier
        long sum = 0;
        //parcours de chaque octet du fichier
        for (byte b : data) {
            sum += b;
        }
        //conversion de la somme en chaîne hexadécimale
        return Long.toHexString(sum); // ou String.valueOf(sum)
    } catch (Exception e) {
        return null;
    }
}
```

5.4 Exploration récursive

L'exploration de l'arborescence utilise un parcours en profondeur (DFS) :

- L'algorithme commence au répertoire de base.
- S'il rencontre un fichier, il l'ajoute à la file partagée (*FileQueue*) pour être traité par des threads d'indexation.
- S'il rencontre un dossier, il entre dans ce dossier et applique récursivement le même traitement sur son contenu.

Ce mécanisme permet donc de parcourir l'ensemble des fichiers présents dans l'arborescence

quelle que soit sa profondeur tout en modifiant progressivement la file de traitement.

```java
private void explorer(File dir) {
    if(dir==null || !dir.isDirectory())
    {
        return;
    }
    //récupère la liste de tous les fichiers/dossiers du répertoire
    File[] elements = dir.listFiles();
    if(elements==null){return;}
    //parcours de chaque éléments du répertoire
    for(File element : elements)
    {
        //si c'est un dossier, on commence la récursivité
        if(element.isDirectory())
        {
            this.explorer(element);
        //si c'est un fichier
        } else if(element.isFile()){
            //récupère l'extension du fichier
            String ext = getExtension(element.getName());
            //si ce fichier a une extension reconnue
            if(estTexte(ext) || estPdf(ext) || estImage(ext))
            {
                //ajout à la file d'attente pour qu'un indexeur le traite
                queue.ajouter(element);
                nbFichiersTrouves++;
            }
        }

    }
}
```

5.5 Communication par pipe

Comme demandé sur le sujet, nous utilisons des pipes pour traiter les fichiers PDF via l'outil système *pdftotext.*

- **ProcessBuilder :** Le programme Java utilise la classe *ProcessBuilder* pour lancer l'exécution de la commande Linux *pdftotext*
- **Mécanisme de pipe :** La sortie standard du programme *pdftotext* est est récupérée sous forme de flux (*InputStream*) ce qui permet de lire directement le texte converti.

Ce fonctionnement est similaire à celui vu en TP en C comme dans tube.c, mais Java simplifie l'utilisation en fournissant des classes de plus haut niveau.

```java
private void indexerPdf(File f, FicheDocument fiche) {
    try {
        // créer un ProcessBuilder pour lancer la commande externe "pdftotext"
        ProcessBuilder pb = new ProcessBuilder(...command: "pdftotext", f.getAbsolutePath(), "-");

        // redirige les erreurs vers la sortie standard
        pb.redirectErrorStream(redirectErrorStream: true);

        // Lancer le processus pdftotext
        Process p = pb.start();
        BufferedReader reader = new BufferedReader(new InputStreamReader(p.getInputStream()));

        String ligne = reader.readLine();
        while(ligne!=null)
        {
            // traite chaque ligne comme indexertexte()
            ligne = ligne.toLowerCase();
            StringTokenizer st = new StringTokenizer(ligne, delim: " ,;.:+-=* /{}?!()[]\\t\\n\\r");
            while(st.hasMoreTokens())
            {
                String mot = st.nextToken();
                if(mot.length()<2) continue;
                if(index.estStopWord(mot)) continue;
                fiche.ajouterMot(mot);
            }
            ligne = reader.readLine();
        }

        //fermer le pipe
        reader.close();

        // attendre que pdftotext ait fini
        p.waitFor();
        //tuer le processus pour libérer les ressources
        p.destroyForcibly();

    } catch (Exception e) {}
}
```

6. Threads et synchronisation

6.1 Liste des threads

Thread	Rôle
Main	CLI utilisateur
Explorateur	Parcours de l'arborescence (producteur)
ThreadIndexeur × 4	Indexation des fichiers (consommateur)
Sauvegarde	Sauvegarde périodique (daemon)
ServeurIndex	Écoute socket TCP (daemon)
ConnexionClient × N	Gère un client distant

6.2 Mécanismes de synchronisation

Comme nous l'avons vu durant ce semestre, pour gérer un mécanisme de multithreading il est impératif de synchroniser leurs accès pour éviter toute corruption de données.

- **synchronized et wait/notify sur FileQueue :**

  Pour éviter de gaspiller des ressources du processeur inutilement, nous avons cherché à toujours vérifier si la file est vide, dès lors les threads indexeurs s'endorment puis se réveillent s'ils reçoivent un nouveau signal qu'un nouveau fichier vient d'être ajouté par l'Explorateur.

```java
public synchronized int ajouter(File f) {
    if(this.list.size()<maxElements)
    {
        this.list.addLast(f);
        notifyAll(); //réveiller les threads qui dormaient en attente
        return 0;
    } else {
        return -1;
    }
}
```

```java
public synchronized File retirer() {
    while(this.list.isEmpty() && this.ferme==false)
    {
        try {
            wait(); //le thread s'endord si la file est vide
        } catch (InterruptedException e) {}
    }
    if(this.list.isEmpty())
    {
        return null;
    }
    return this.list.removeFirst();
}
```

6.3 Modèle producteur/consommateur

Notre projet repose sur un modèle de producteur/consommateur notamment avec l'Explorateur (le producteur) et les indexeurs (les consommateurs) où on a le schéma de fonctionnement suivant :

- L'**Explorateur** appelle *queue.ajouter(f)*, ce qui déclenche *notifyAll()*.
- Les **ThreadIndexeur** (x4) appellent *queue.retirer()*, ce qui déclenche *wait()* si la file est vide.

Nous pouvons bien sûr faire le lien avec le cours, notamment où nous avons observé un comportement identique dans le problème du *SleepingBarber* :

- **L'Explorateur** (*SimulArriveeClient*) génère des fichiers.
- **FileQueue** (*SalleAttente*) gère l'espace disponible et réveille le personnel quand un fichier arrive.
- **ThreadIndexeurs** (*Barber*) dorment s'il n'y a pas de travail, et traitent les fichiers un par un dès qu'ils sont réveillés.

Par exemple, si l'Explorateur trouve 100 fichiers en 10 secondes mais qu'un seul ThreadIndexeur les traite, il faudra 100 fois plus de temps. Avec 4 ThreadIndexeur, on divise donc, en théorie, le temps par 4.

7. Jeux de tests

Cette section présente les scénarios de test utilisés pour valider le fonctionnement de l'application.

7.1 Test 1 : Indexation d'un répertoire

```
>index C:\Users\Mehdi\Desktop\INDEXEUR\Indexeur_info4B-main\fi\src\tests
Exploration démarée.
Exploration terminée, 6fichiers trouvés
Index sauvegardé
Indexation terminée.
```

7.2 Test 2 : Recherche par mots-clés

```
>rechercher test
[0.4054651081081644]
C:\Users\Mehdi\Desktop\INDEXEUR\Indexeur_info4B-main\fi\src\tests\testTxt.txt
[0.4054651081081644]
C:\Users\Mehdi\Desktop\INDEXEUR\Indexeur_info4B-main\fi\src\tests\testTxtDoublo
n.txt
[2.9813610890306203E-4]
C:\Users\Mehdi\Desktop\INDEXEUR\Indexeur_info4B-main\fi\src\tests\sujetProjetIn
fo4B.pdf
[2.9813610890306203E-4]
C:\Users\Mehdi\Desktop\INDEXEUR\Indexeur_info4B-main\fi\src\tests\sujetProjetIn
fo4B Doublon.pdf
```

7.3 Test 3 : Détection de doublons

```
>doublons
[C:\Users\Mehdi\Desktop\INDEXEUR\Indexeur_info4B-main\fi\src\tests\testimagedou
blon.jpg,
C:\Users\Mehdi\Desktop\INDEXEUR\Indexeur_info4B-main\fi\src\tests\testimage.jpg
]
[C:\Users\Mehdi\Desktop\INDEXEUR\Indexeur_info4B-main\fi\src\tests\sujetProjetI
nfo4B.pdf,
C:\Users\Mehdi\Desktop\INDEXEUR\Indexeur_info4B-main\fi\src\tests\sujetProjetIn
fo4B Doublon.pdf]
[C:\Users\Mehdi\Desktop\INDEXEUR\Indexeur_info4B-main\fi\src\tests\testTxt.txt,
C:\Users\Mehdi\Desktop\INDEXEUR\Indexeur_info4B-main\fi\src\tests\testTxtDoublo
n.txt]
```

7.4 Test 4 : Lister les documents indexés

```
> list
[..jpg] testimagedoublon.jpg (22.212890625 Ko, 0 termes)
[..txt] testTxt.txt (0.015625 Ko, 1 termes)
[..txt] testTxtDoublon.txt (0.015625 Ko, 1 termes)
[..jpg] testimage.jpg (22.212890625 Ko, 0 termes)
[..pdf] sujetProjetInfo4B Doublon.pdf (187.380859375 Ko, 1360 termes)
[..pdf] sujetProjetInfo4B.pdf (187.380859375 Ko, 1360 termes)
```

7.5 Test 5 : Annotation et recherche par métadonnées

```
> annoter
C:\Users\Mehdi\Desktop\INDEXEUR\Indexeur_info4B-main\fi\src\tests\testimagedoub
lon.jpg sujet papillon
Annotation ajoutée.
>annoter
C:\Users\Mehdi\Desktop\INDEXEUR\Indexeur_info4B-main\fi\src\tests\testimage.jpg
sujet papillon
Annotation ajoutée.

> meta sujet papillon
[1.0]
C:\Users\Mehdi\Desktop\INDEXEUR\Indexeur_info4B-main\fi\src\tests\testimagedoub
lon.jpg
[1.0]
C:\Users\Mehdi\Desktop\INDEXEUR\Indexeur_info4B-main\fi\src\tests\testimage.jpg
```

7.6 Test 6 : Status

```
> status
Fichiers : 6
Termes   : 671
En cours : false
```

7.7 Test 7 : Ajout de mots personnalisés

```
> rechercher if
Aucun résultat.

> addterme if
Terme ajouté.

>index C:\Users\Mehdi\Desktop\INDEXEUR\Indexeur_info4B-main\fi\src\tests
Exploration démarée.
Exploration terminée, 6fichiers trouvés
Index sauvegardé
Indexation terminée.

> rechercher if
[0.27465307216702745]
C:\Users\Mehdi\Desktop\INDEXEUR\Indexeur_info4B-main\fi\src\tests\testTxt.txt
```

```
[0.27465307216702745]
C:\Users\Mehdi\Desktop\INDEXEUR\Indexeur_info4B-main\fi\src\tests\testTxtDoublo
n.txt
```

7.8 Test 8 : Ajout de stop-words

```
> rechercher test
[0.1013662770270411]
C:\Users\Mehdi\Desktop\INDEXEUR\Indexeur_info4B-main\fi\src\tests\testTxt.txt
[0.1013662770270411]
C:\Users\Mehdi\Desktop\INDEXEUR\Indexeur_info4B-main\fi\src\tests\testTxtDoublo
n.txt
[2.9813610890306203E-4]
C:\Users\Mehdi\Desktop\INDEXEUR\Indexeur_info4B-main\fi\src\tests\sujetProjetIn
fo4B.pdf
[2.9813610890306203E-4]
C:\Users\Mehdi\Desktop\INDEXEUR\Indexeur_info4B-main\fi\src\tests\sujetProjetIn
fo4B Doublon.pdf

> addstop test
Stop-word ajouté.

> rechercher test
Aucun résultat.
```

7.9 Test 9 : Connexion réseau

```
IP du serveur : xxx.xxx.xxx.xxx
===== INDEXEUR DE FICHIERS =====
RECHERCHER <mots>                | recherche TF-IDF
META <clé> <valeur>              | recherche par métadonnée
ANNOTER <chemin> <clé> <val>     | annoter un fichier
ADDSTOP <mot>                    | ajouter stop-word
ADDTERME <mot>                   | ajouter terme perso
LIST                             | lister les fichiers
STATUS                           | état de l'indexeur
DOUBLONS                         | détecter les doublons
RECUPERER <chemin> <dest>        | récupérer un fichier
QUIT                             | quitter

> rechercher test
[0.13515503603605478]
C:\Users\Mehdi\Desktop\INDEXEUR\Indexeur_info4B-main\fi\src\tests\testTxt.txt
[0.13515503603605478]
C:\Users\Mehdi\Desktop\INDEXEUR\Indexeur_info4B-main\fi\src\tests\testTxtDoublo
n.txt
[2.9813610890306203E-4]
C:\Users\Mehdi\Desktop\INDEXEUR\Indexeur_info4B-main\fi\src\tests\sujetProjetIn
fo4B.pdf
[2.9813610890306203E-4]
C:\Users\Mehdi\Desktop\INDEXEUR\Indexeur_info4B-main\fi\src\tests\sujetProjetIn
fo4B Doublon.pdf
```

```
> annoter
C:\Users\Mehdi\Desktop\INDEXEUR\Indexeur_info4B-main\fi\src\tests\sujetProjetIn
fo4B.pdf annee 2026
Annotation ajoutée.

> meta annee 2026
[1.0]
C:\Users\Mehdi\Desktop\INDEXEUR\Indexeur_info4B-main\fi\src\tests\sujetProjetIn
fo4B.pdf

> addstop test
Stop-word ajouté : test

> rechercher test
Aucun résultat

> list
[..pdf] sujetProjetInfo4B.pdf (187.380859375 Ko, 1360 termes)
[..txt] testTxt.txt (0.03125 Ko, 3 termes)
[..jpg] testimage.jpg (22.212890625 Ko, 0 termes)
[..jpg] testimagedoublon.jpg (22.212890625 Ko, 0 termes)
[..pdf] sujetProjetInfo4B Doublon.pdf (187.380859375 Ko, 1360 termes)
[..txt] testTxtDoublon.txt (0.03125 Ko, 3 termes)

> doublons
Groupe :
[C:\Users\Mehdi\Desktop\INDEXEUR\Indexeur_info4B-main\fi\src\tests\testimagedou
blon.jpg,
C:\Users\Mehdi\Desktop\INDEXEUR\Indexeur_info4B-main\fi\src\tests\testimage.jpg
]
Groupe :
[C:\Users\Mehdi\Desktop\INDEXEUR\Indexeur_info4B-main\fi\src\tests\testTxt.txt,
C:\Users\Mehdi\Desktop\INDEXEUR\Indexeur_info4B-main\fi\src\tests\testTxtDoublo
n.txt]
Groupe :
[C:\Users\Mehdi\Desktop\INDEXEUR\Indexeur_info4B-main\fi\src\tests\sujetProjetI
nfo4B.pdf,
C:\Users\Mehdi\Desktop\INDEXEUR\Indexeur_info4B-main\fi\src\tests\sujetProjetIn
fo4B Doublon.pdf]

> recuperer
C:\Users\Mehdi\Desktop\INDEXEUR\Indexeur_info4B-main\fi\src\tests\testTxt.txt
C:\Users\Mehdi\Desktop\recup.txt
Fichier récupéré : C:\Users\Mehdi\Desktop\recup.txt

> quit
Au revoir !
```

8. Répartition des tâches

Tâche	BENSELAMA	BEKHTAOUI
Analyse fonctionnelle	X	----------
Couche 1 : FileQueue, IndexInverse	X	X
Couche 1 : FicheDocument, sérialisation	X	X
Couche 2 : Explorateur, ThreadIndexeur	----------	X
Couche 2 : Pipes (pdftotext)	----------	X
Couche 3 : Serveur, Client	X	----------
Couche 4 : Interface CLI	X	X
Tests et débogage	X	X
Rédaction du rapport	X	X

9. Conclusion

En conclusion, ce projet nous a permis de mettre en pratique les nombreux concepts théoriques vus en cours, tels que la gestion de la concurrence avec un système de synchronisation, la conception d'une application multi-threadée, la communication inter-processus via des pipes, l'utilisation de sockets TCP, la sérialisation d'objets, et surtout la conception en couches afin de s'approcher davantage des bonnes pratiques de l'ingénierie logicielle.

En ce qui concerne les difficultés rencontrées, la structure du projet en couche par couche était compliquée à cerner au début, sans oublier les gestions de problèmes de concurrence et la communication par pipe avec ProcessBuilder étaient difficiles à appréhender.

Bien sûr, l'organisation des tâches entre le binôme constituait un obstacle en plus à franchir mais l'utilisation de github a grandement aidé à s'organiser efficacement.
De plus, pour les améliorations possibles, une interface graphique en Swing ou JavaFX serait vraiment un plus pour l'expérience utilisateur où on aurait pu étendre les fonctionnalités avec un système de gestion dynamique de fichiers et surtout implémenter un système de cache pour éviter de ré-indexer à chaque fois les fichiers non modifiés.

10. Annexe : Utilisation d'un LLM

10.1 Code généré par le LLM

Pour la génération du code on demandait au LLM de générer / corriger du code en lui donnant la classe (le fichier .java) en annexe avec la signature de la méthode

```
-IndexInverse.java : rechercher() ligne 120.
-IndexInverse.java : initStopWords() ligne 25
-ProgrammeClient.java : boucleCommandes() case "RECUPERER" ligne 95
-Client.java : traiterCommande(String ligne)case "RECUPERER" ligne 127
- ThreadIndexeur : ligne 97 et 98 p.waitFor() et p.destroyForcibly()
```

10.2 Texte du rapport généré par le LLM

Le rapport a été entièrement rédigé par le binôme, la structure néanmoins (cf. Table des matières et les parties) ont été générées par le LLM pour structurer le plan de rédaction, aucun passage n'a été copié/collé directement depuis le LLM.

10.3 Prompts utilisés

```
- Quelles sont les principales méthodes de la classe File nécessaires pour coder un indexeur de fichiers
- Comment faire en sorte qu'un thread s'arrête automatiquement quand le programme principal se termine
- Initialise une liste de stop-words en Java
- Comment envoyer un fichier en utilisant un socket texte en Java
- Comment récupérer le fichier du côté du client
- Comment fonctionne le principe de pipe avec la classe ProcessBuilder en Java
-Comment utiliser le pdftotext en java
-Comment utiliser le StringTokenizer
-Comment découper un String en ligne de commande pour pouvoir traiter les parties séparément
```

10.4 Ce que nous avons appris du LLM

Le LLM expliquait certains concepts avec des exemples concrets comme par exemple pour le modèle Producteur/Consommateur où il utilise l'analogie du *SleepingBarber* : un barbier dort quand il n'y a pas de clients et se réveille quand un client arrive et traite le client.

- **Les concepts découverts :**
  - Le mot clé ***continue*** pour sauter à la prochaine itération dans un boucle (exemple pour ignorer les stop-words et passer au mot suivant sans exécuter le reste du bloc)
  - L'interface **Comparable** pour implémenter le tri des résultats de recherche par score décroissant
  - La classe **Byte** pour la somme des ***bytes*** pour calculer un checksum (additionner tous les octets et convertir en hexadécimal grâce à la méthode toHexString()
  - La classe **ProcessBuilder** pour lancer le programme externe *pdftotext* et récupérer sa sortie via un pipe (getInputStream())
  - La classe **Base64** pour encoder les bytes d'un fichier en texte pour l'envoyer via un socket texte
  - La méthode **setDaemon(true)** pour que le serveur et le thread de sauvegarde s'arrêtent automatiquement quand le programme principal se termine
  - La classe **StringTokenizer** qui permet de prendre une chaîne de caractères et de la découper en petits morceaux selon des délimiteurs

Le LLM corrigeait également certaines erreurs de syntaxe comme des while mal placés, l'oublie de break dans le switch et expliquait pourquoi c'était faux.